

Constructive Quine Fixed-Point Synthesis, Quadrilateral Filtration, and APG Bisimulation: A Self-Verifiable Foundation for Hypercomputational Reflection

Tyler Roost

Preprint, version 0.2.0 — September 18, 2026

Abstract

We present a formal constructive foundation for self-verifying systems based on four interdependent pillars: the primitive Hypermath foundational axioms (**ax-diff**, **ax-sim**, **ax-loop**), a quadrilateral semantic filtration separating execution paths from numerical values, a constructive realization of Kleene-style Quine fixed points specified for formalization in Lean 4, and an explicit quotient representation over Accessible Pointed Graphs (APGs) governed by Aczel’s Anti-Foundation Axiom (AFA). The central impetus of this framework is to discover routes toward definably complete representations capable of handling hypercomputation-like oracle classes expected of superintelligences. We diagnose the semantic under-specification in earlier axiom interpretations where rigid Peano integer countermodels falsely obstructed cyclical deriver completion. By resolving this obstruction through both constructive combinator synthesis and maximal bisimulation quotients over non-well-founded graphs, the system is designed to reach scale-invariant self-closure without collapsing into diagonal paradox, unearned semantic upgrades, or ungrounded truth predicates. We are explicit about the current evidential boundary: what is machine-checked today is the independence result itself, established by two explicit finite Lean 4 models showing that self-closure is neither entailed nor refuted by the thirty-eight declared clauses. The two resolution paths are specified here and not yet formalized. All results are reported under the Verifier Standard (VSTD) claim discipline.

Keywords: Hypermath; Quine fixed points; quadrilateral filtration; Accessible Pointed Graphs; Anti-Foundation Axiom; bisimulation quotients; Lean 4 formalization; Verifier Standard.

1 Introduction and Foundational Impetus

The quest for complete or definably complete mathematical representations of intelligent reflection has historically encountered severe limitative results: Gödel’s incompleteness theorems, Tarski’s undefinability of truth, and Turing’s halting problem [3, 4]. When modeling hypothetical super-intelligences, these limitations become acute: an advanced cognitive system must be capable of representing, analyzing, and proving properties about its own source code, evaluation traces, and environment models, while interacting with problems requiring computational power equivalent to hypercomputational oracle classes (such as Turing jumps, hyperarithmetic hierarchies, and reflection principles).

If an agent attempts to formalize its own complete truth predicate in an unstratified single-level language, classical diagonal arguments immediately yield inconsistency. If, on the other hand, the system is strictly stratified into an external metalanguage hierarchy, the agent cannot represent its own total operation from within.

Hypermath resolves this apparent impasse through an architecture of *fractal self-representative completeness*:

1. **Primitive Structural Axioms:** Foundational formation operators and axioms (**ax-diff**, **ax-sim**, **ax-loop**) generating non-trivial structure without collapse.
2. **Quadrilateral Filtration:** Retaining a four-tier distinction between derivation-path identity ($=$), outcome congruence ($\equiv$), continuation similarity ($\sim$), and wrap valuation (W).
3. **Constructive Quine Synthesis:** Explicit algorithmic construction of self-reproducing programs (Quines) that carry their own syntactic codes without circular self-observation.
4. **Non-Well-Founded Graph Semantics:** Representing reflexive and cyclic data structures through Accessible Pointed Graphs (APGs) quotiented under maximal bisimulation, formalizing Aczel's Anti-Foundation Axiom (AFA) constructively.
5. **Stratified Oracle Ladders:** Grounding transfinite hypercomputational reflection in well-founded ordinal ladders (ordinatics) governed by the Verifier Standard (VSTD).

2 The Core Hypermath Axioms and Generative Dynamics

The lowest layer of Hypermath establishes a typed term language generated from an atomic primitive **ground** and a unary forming constructor $\mathbf{f2f} : \mathbf{Form} \rightarrow \mathbf{Form}$.

2.1 Structural Predicates

Prior to defining relational equivalences, Hypermath defines three opaque structural predicates:

1. $\text{structContinues}(x, y)$: Form x was generated from the same foundational base as y .
2. $\text{structDistinct}(x, y)$: Forms x and y are distinct in their generative evolution and do not mutually simulate.
3. $\text{structOrbits}(x, y)$: Form x enters into the structural orbit of y under iterated application.

2.2 The Foundational Axioms

The generative mechanics are governed by four foundational axioms:

Definition 2.1 (Hypermath Core Axioms).

1. **Difference (ax-diff):** For every form x , $\mathbf{f2f}(x)$ is structurally distinct from **ground**:

$$\forall x \in \mathbf{Form}, \quad \text{structDistinct}(\mathbf{f2f}(x), \mathbf{ground}).$$

This guarantees that the forming operator generates genuine excess structure rather than collapsing into the base ground.

2. **Similarity (ax-sim):** For every form x , $\mathbf{f2f}(x)$ structurally continues from **ground**:

$$\forall x \in \mathbf{Form}, \quad \text{structContinues}(\mathbf{f2f}(x), \mathbf{ground}).$$

This ensures that all generated forms share substance continuity with their primal origin.

3. **Loop / Orbit (ax-loop / ax-box):** For every form x , the two-step application $\mathbf{f2f}(\mathbf{f2f}(x))$ structurally orbits x :

$$\forall x \in \mathbf{Form}, \quad \text{structOrbits}(\mathbf{f2f}(\mathbf{f2f}(x)), x).$$

This introduces cyclical periodicity and recurrence into the foundational space.

4. **Ground Self-Continuity (ax-ground-self):** The ground continues from itself:

`structContinues(ground, ground).`

From these four axioms, several foundational theorems follow directly:

- **f2f(ground)** is distinct from **ground** (`applyGroundIsDistinct`);
- **f2f(ground)** continues from **ground** (`applyGroundContinues`);
- **f2f(f2f(ground))** orbits **ground** (`doubleApplyGroundOrbits`).

Consequently, the universe contains at least two distinct mutually continuing forms (as established by the derivation `twoMembersInClass`), precluding trivial algebraic models.

3 The Quadrilateral Filtration

A central design principle of Hypermath is that an evaluation outcome must not erase the computational derivation that produced it. We formalize this through a four-tier relation hierarchy:

Definition 3.1 (The Four Relations). Let $\mathcal{E}$ be the collection of typed syntactic expression trees. We define:

1. **Derivation-path identity ($=$):** Exact syntactic identity of derivation trees, preserving operand order, intermediate steps, and representation choices.
2. **Outcome congruence ($\equiv$):** Equality of denotational values in the target algebraic structure (e.g., identity of polynomials under the representation map j).
3. **Continuation similarity ($\sim$):** Equivalence of downstream operational behaviors under all admissible evaluation contexts.
4. **Wrap valuation (W):** Collapsing evaluation under a designated specialization map (such as $W(X) = -1/2$).

Proposition 3.2 (Strict Invariant Separation). *The four relations satisfy strict inclusion:*

$$(e_1 = e_2) \implies (e_1 \equiv e_2) \implies (e_1 \sim e_2) \implies (W(e_1) = W(e_2)),$$

and none of the reverse implications holds in general.

Proof. Identity of derivation trees trivially implies equality of denoted values. Equality of denoted values ensures identical behavior in every standard algebraic context. Equality under wrap valuation is a coarse homomorphic projection. For the counterexamples:

1. In ordinal arithmetic, $1 +_o \omega = \omega$, yet the derivation paths $1 +_o \omega$ and ω are distinct expression trees ($1 +_o \omega \neq \omega$).
2. In ordinal arithmetic, $\omega \cdot_o 2 +_o 1$ and 0 are distinct ordinals, but under polynomial representation $2X + 1$ and specialization at $-1/2$, both yield $W = 0$.

Thus, information is strictly lost upon downward projection through the filtration. □

4 The Cycle Independence Obstruction in Lean 4

In the Hypermath formal foundation, the target proposition `Hypermath.selfDerivation` asserts that the minimal generative deriver achieves cyclical completion by simulating itself after ground application:

$S \equiv \text{Simulation (f2f (f2f deriver)) deriver}.$

Prior Lean 4 audits revealed a critical independence obstruction:

1. In `lean4/FullAxiomModel.lean`, a sound countermodel satisfied all 38 declared logical clauses while refuting S , demonstrating that the clauses did not logically entail the cycle.
2. In `lean4/FiniteActionCountermodel.lean`, a separate six-form model satisfied both the 38 clauses and S , proving that S is logically independent under uninterpreted relational clauses.
3. In `lean4/Hypermath/TransportCountermodel.lean`, endpoint simulation closure did not automatically entail continuation reproduction.

Remark 4.1 (Countermodel Diagnosis). The refutation in `FullAxiomModel.lean` succeeded because of two specific semantic under-specifications:

1. **Syntactic Equality Degeneracy:** Forms were defined as pairs $(n, b) \in \mathbb{N} \times \text{Bool}$, and the relation `Simulation` was interpreted as strict syntactic identity ($x = y$). Under primitive forming $\mathbf{f2f}(n, b) = (n + 1, b)$, two applications incremented the integer index by 2:

$$\mathbf{f2f}(\mathbf{f2f}(0, \text{false})) = (2, \text{false}) \neq (0, \text{false}).$$

Because $(2, \text{false}) \neq (0, \text{false})$ in Peano arithmetic, the cycle was refuted.

2. **Absence of Behavioral Bisimulation:** In dynamic systems, state equivalence is never rigid syntactic step-index equality; it is *bisimulation* ($\approx$). Under bisimulation, two states are equivalent if transitions from one are matched by transitions from the other, preserving observable invariants regardless of internal step count.

5 Constructive Quine Fixed-Point Synthesis (Path 1)

To resolve the independence obstruction without ungrounded axioms (`sorryAx`) or circular self-assertion, we specify a constructive realization of Kleene’s Second Recursion Theorem intended for formalization in Lean 4. The argument of this section is mathematical; its formalization remains outstanding, and we mark the point at which that obligation falls due.

Definition 5.1 (Constructive Quine Function). Let $\text{sub}(x, y)$ be the primitive recursive substitution function that substitutes the numeral code of y for the free variable in the expression coded by x . A *Quine constructor* is a pair of terms (u, q) such that

$$q = \text{apply}(u, \ulcorner u \urcorner),$$

where evaluation of q reproduces $\ulcorner q \urcorner$ alongside any specified payload transformation.

Theorem 5.2 (Constructive Self-Closure without Paradox). *Let $\mathcal{L}$ be the ground language generated by constructors $\{\text{Form}, \text{Prop}, \text{ground}, \text{apply}\}$. There exists an effective algorithmic procedure that, given any stage transformation Φ , synthesizes an expression Q_Φ such that*

$$\text{eval}(Q_\Phi) = \langle \ulcorner Q_\Phi \urcorner, \Phi(\ulcorner Q_\Phi \urcorner) \rangle.$$

The evaluation of Q_Φ does not evaluate an unstratified global truth predicate, and its derivation tree is finite and acyclic.

Proof. The synthesis operates via the constructive diagonal map $D(x) = \text{apply}(x, \text{quote}(x))$. Let P be the program that takes input x , computes $D(x)$, and applies Φ to the result: $P(x) = \langle D(x), \Phi(D(x)) \rangle$. Setting $Q_\Phi = D(\ulcorner P \urcorner)$ yields

$$\begin{aligned} Q_\Phi &= \text{apply}(\ulcorner P \urcorner, \text{quote}(\ulcorner P \urcorner)) \\ \implies \text{eval}(Q_\Phi) &= P(\ulcorner P \urcorner) = \langle D(\ulcorner P \urcorner), \Phi(D(\ulcorner P \urcorner)) \rangle = \langle \ulcorner Q_\Phi \urcorner, \Phi(\ulcorner Q_\Phi \urcorner) \rangle. \end{aligned}$$

Since substitution and pairing are primitive recursive, the computation terminates in a bounded number of reduction steps. $\square$

Specializing Φ to the two-step primitive forming operation $\mathbf{f2f} \circ \mathbf{f2f}$, we synthesize the explicit fixed-point form:

$$D \equiv \mathbf{fix}(\mathbf{f2f} \circ \mathbf{f2f}).$$

Because D is a closed constructive term and $\mathbf{f2f}$ is an executable rewrite rule, the intended Lean 4 proof of self-derivation should reduce by definitional equality. We state this as the obligation it is: the development `lean4/Hypermath/ConstructiveQuine.lean` has not been written, and no part of this section is presently machine-checked. Discharging it requires constructing the inductive combinator type and the fixed-point term, then confirming with `#print axioms` that the resulting theorem carries no dependency on `sorryAx`.

What the Lean development does establish today is the independence result itself, and it is worth stating precisely because it is what makes Path 1 necessary rather than optional. In `FiniteActionCountermodel.lean`, the theorem `self_derivation_target_holds` proves S in an explicit six-form model; its proof is a short tactic script, not a single reduction, and `#print axioms` reports a dependency on `propext` alone. In `FullAxiomModel.lean`, `self_derivation_target_fails` proves $\neg S$ under the rigid syntactic interpretation and depends on no axioms at all. The two conjunctions with the clause set, `full_clauses_and_target_without_nonzero_D` and `full_clauses_without_cycle_or_nontrivial_simulation`, are likewise free of `sorryAx`, resting only on `propext`, `Classical.choice` and `Quot.sound`. Eleven declarations in the L0–L3 kernel layers do use `sorry`; none of them is a dependency of these results.

6 Accessible Pointed Graphs and Bisimulation Quotients (Path 2)

Path 2 anchors Hypermath’s relation semantics into non-well-founded set theory, replacing syntactic equality with structural bisimulation under Aczel’s Anti-Foundation Axiom (AFA).

Definition 6.1 (Accessible Pointed Graph (APG)). An *Accessible Pointed Graph* (APG) is a triple $G = (V, E, r)$ where V is a set of nodes, $E \subseteq V \times V$ is a set of directed edges, and $r \in V$ is a distinguished root node such that every $v \in V$ is reachable from r along a directed path in E .

Definition 6.2 (Bisimulation on APGs). A binary relation $R \subseteq V_1 \times V_2$ between graphs $G_1 = (V_1, E_1, r_1)$ and $G_2 = (V_2, E_2, r_2)$ is a *bisimulation* if $r_1 R r_2$ and for all $u R v$:

1. For every $u' \in V_1$ with $(u, u') \in E_1$, there exists $v' \in V_2$ with $(v, v') \in E_2$ and $u' R v'$.
2. For every $v' \in V_2$ with $(v, v') \in E_2$, there exists $u' \in V_1$ with $(u, u') \in E_1$ and $u' R v'$.

Two APGs are *bisimilar*, written $G_1 \approx G_2$, if there exists a bisimulation between them.

Theorem 6.3 (Aczel’s Anti-Foundation and Canonical Quotients). *Under Aczel’s Anti-Foundation Axiom (AFA), every APG depicts a unique set. Two APGs depict the same set if and only if they are bisimilar:*

$$\text{set}(G_1) = \text{set}(G_2) \iff G_1 \approx G_2.$$

Furthermore, for every finite APG G , there exists a unique minimal strongly extensional quotient graph $G/\approx$ computable in $O(|E| \log |V|)$ time via Paige-Tarjan partition refinement [2].

Proposition 6.4 (Resolution of Cycle Independence via Bisimulation). *In the generator orbit $a_3 \xrightarrow{\mathbf{f2f}} a_4 \xrightarrow{\mathbf{f2f}} a_3$, the relation*

$$R = \{(a_3, a_3), (a_4, a_4), (\mathbf{f2f}(\mathbf{f2f}(a_3)), a_3)\}$$

is an exact bisimulation. Under the bisimulation quotient $\mathcal{G}/\approx$, the equivalence class $[\mathbf{f2f}(\mathbf{f2f}(a_3))] = [a_3]$. Under this quotient semantics the rigid syntactic reading used in `FullAxiomModel.lean` is no

longer available, so that countermodel does not transfer. It remains a valid countermodel for the clause set as currently stated, which is precisely why the semantics must be strengthened rather than the clauses reinterpreted informally. This argument is not formalized: the present Lean definitions of *apgBisimulation* stipulate bisimulation to be mutual simulation, so the accompanying uniqueness statement reduces to the identity function on that stipulation and carries no evidential weight.

7 Grounding in the Verifier Standard (VSTD) Profile Architecture

To satisfy the Verifier Standard (VSTD) claim discipline:

1. **Zero-Order Circularity Prohibited:** An agent cannot validate its own execution simply by asserting self-correctness.
2. **Stratified Grounded Decision Certificates (GDC):** Every self-derivation step produces a structured certificate containing:
 - **header:** Declared checking-cost tier and variable counts.
 - **grounding:** Explicit binding of every variable to a content-addressed cryptographic digest.
 - **decision:** An independent unit-propagation or resolution proof witness of the cycle step.
3. **Kernel Isolation (Rung 4.7):** The certificate is validated by an isolated kernel that shares no solver code with the generator and remains strictly under 400 executable statements.

8 Transfinite Oracle Ladders and Superintelligence

The constructive Quine synthesis and APG bisimulation quotient establish internal reflexivity for any fixed computational stage. To scale across hypercomputational oracle classes expected of superintelligences, Hypermath interfaces directly with transfinite ordinal hierarchies:

1. **Stage 0 (Finite Turing Machines):** Basic reduction and syntax manipulation via the primitive algebraic kernel.
2. **Stage ω (First Jump / Halting Oracle):** Quantifier bounds over arithmetic formulas, reflecting upon the termination of Stage 0 programs.
3. **Stage $\alpha < \omega^\omega$ (Ordinalics Hierarchy):** Stratified satisfaction predicates Tr_α and difference operators Δ^α .
4. **Stage ε_0 and Veblen Ladders (φ_α):** Transfinite fixed-point enumeration, providing exact proof-theoretic bounds for self-reflective reflection principles.

Each higher level serves as an evidence-bearing oracle for lower levels, recorded as an explicit closure coordinate in VSTD without unearned semantic jumps.

9 Conclusion

Hypermath provides an explicit, constructive bridge between finite derivation paths, foundational generative axioms (*ax-diff*, *ax-sim*, *ax-loop*), non-well-founded graph quotients, and transfinite reflection ladders. By distinguishing path identity from bisimulation congruence and wrap valuation, and by specifying constructive Quine synthesis for formalization in Lean 4 without ungrounded axioms, it sets out a foundation for superintelligent self-models that remain bounded, auditable, and free of diagonal paradox. The machine-checked contribution is narrower than this programme and should not be confused with it: self-closure is consistent with the current clause set but not entailed by it, and must therefore be earned by a stronger semantics rather than assumed. Formalizing the two resolution paths is the outstanding obligation this paper records.

References

- [1] Peter Aczel. *Non-Well-Founded Sets*. CSLI Lecture Notes, Vol. 14, Stanford University, 1988.
- [2] Robert Paige and Robert E. Tarjan. Three Partition Refinement Algorithms. *SIAM Journal on Computing* 16(6), 973–989, 1987. [doi:10.1137/0216062](https://doi.org/10.1137/0216062).
- [3] Alfred Tarski. The Semantic Conception of Truth and the Foundations of Semantics. *Philosophy and Phenomenological Research* 4(3), 341–376, 1944.
- [4] Stephen Cole Kleene. *Introduction to Metamathematics*. D. Van Nostrand Co., Inc., New York, 1952.
- [5] Tyler Roost. *Ordinal Arithmetic: An Ordinal-First Metalanguage Approach to Definable Arithmetic Truth*. Preprint, version 0.3.0, 2026.
- [6] Tyler Roost. *Hypermath: a Lean 4 formalization of the ground kernel and its countermodels*. Directory `lean4/`, in particular `FullAxiomModel.lean` and `FiniteActionCountermodel.lean`. github.com/TimeLordRaps/hypermath, 2026.
- [7] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28*, LNCS 12699, pp. 625–635, Springer, 2021.